Topic: Pipeline Hazards

Q. Define Pipeline Hazards. Classify and explain the three main types of pipeline hazards (Structural, Data, and Control) and briefly mention methods to handle them.

> 📌 **Definition to Remember**
> A **Pipeline Hazard** occurs when the next instruction in a pipeline cannot execute in its designated clock cycle. This stalls the pipeline (creates a "bubble"), preventing it from achieving maximum instruction throughput.

---

1. Structural Hazards

Occurs when two instructions in the pipeline need the **same hardware resource** at the exact same time (Resource Conflict).
* **Example:** Instruction 1 tries to write to memory while Instruction 4 tries to fetch from memory, but there is only one memory port.
* **Handling:**
* **Resource Duplication:** Split cache into separate Data Cache and Instruction Cache (Harvard Architecture).
* **Stalling:** Delay one instruction by inserting an empty cycle (bubble).

2. Data Hazards

Occurs due to **Data Dependency**, meaning an instruction needs the result of a previous instruction that hasn't finished executing yet.
* **Example:**
`I1: ADD R1, R2, R3` (Calculates R1)
`I2: SUB R4, R1, R5` (Needs R1)
If I2 reaches the EX stage before I1 writes to R1, I2 uses old, incorrect data.
* **Handling:**
* **Data Forwarding (Bypassing):** Hardware routes the ALU output of I1 directly back to the ALU input for I2, skipping the register write phase.
* **Stalling:** Pause I2 until I1 writes the data.
* **Compiler Scheduling:** Compiler reorders instructions to put independent tasks between I1 and I2.

3. Control (Branch) Hazards

Occurs due to **Branch instructions** (Jump, If-Else). The pipeline fetches instructions sequentially. When a branch is encountered, the target address isn't known until the execute stage. Meanwhile, wrong instructions may have been fetched.
* **Handling:**
* **Pipeline Flushing:** If the branch is taken, flush (delete) the wrongly fetched instructions (causes a time penalty).
* **Branch Prediction:** Hardware predicts if the branch will be taken. If wrong, flush. If right, no penalty.
* **Delayed Branching:** Compiler places a useful, independent instruction immediately after the branch.

⭐ **Must-Write Points (for 10 marks)**

1. **Pipeline Hazard**: A situation preventing the next instruction from executing in its proper cycle.
2. Three Types: **Structural, Data, and Control Hazards**.
3. **Structural Hazard**: Resource conflict (two instructions need the same hardware). Fixed by duplicating resources (Instruction vs Data cache).
4. **Data Hazard**: Data dependency (Instruction needs data from an unfinished prior instruction).
5. Data Hazards are fixed via **Data Forwarding** or compiler scheduling.
6. **Control Hazard**: Caused by branch/jump instructions fetching the wrong sequential path.
7. Control Hazards are fixed via **Branch Prediction**, flushing, or delayed branching.

⚡ **Quick Recall**

`Pipeline Hazards → Structural (Hardware conflict → fix: duplicate resources) → Data (Dependency → fix: Data Forwarding/Bypass) → Control (Branching → fix: Branch Prediction/Flushing)`